

4A. Práctica. Sistema de Gestión de Trámites Ciudadanos

Eric Contreras Haro

Jueves, 27 de agosto de 2026

Grupo 02, Generación 14

INTRODUCCIÓN

En esta practica realizaremos el análisis, desarrollo e implementación de un sistema de gestión de trámites ciudadanos. En el cual, una persona podrá consultar el sistema para realizar alguno de los trámites ofertados, como son:

- Constancias de residencia.
- Licencias.
- Permisos para conducir vehículos.
- Autorizaciones para comercio en vía pública.
- Certificados.
- Reposiciones de documentos oficiales.
- Solicitudes de apoyos.
- Etc.

El proceso que se mapea debe contemplar la ruta de trabajo de cada uno de los trámites referidos. Es decir, aunque algunos procesos compartan el mismo flujo, deben de identificarse las diferencias, para poderlas aplicar de una forma exclusiva de la actividad.

El problema con el sistema actual es que llevan sus registros en hojas de cálculo y en documentos físicos. Lo cual hace que los trámites se demoren o sean difíciles de realizar, tanto para los usuarios como para los servidores públicos.

Estás dificultades son punto que podemos ocupar como área de oportunidades para el sistema que se va a implementar. Nos permiten modelar/diseñar módulos, funcionalidades y/o procesos, que permitan realizar de una manera eficiente los trámites.

Para el ejercicio de este trabajo, se tomará como punto de partida un trámite: la licencia de conducir. Se debe de considerar a los actores, procesos involucrados con el objetivo de abstraerlos al P.O.O. para diseñar un sistema que cubra todos los requisitos, de tal manera que el resultado sea en óptimos, escalable y con tendencias a mejorar.

DESARROLLO

Problemática principal: La gestión en la dependencia se realiza actualmente mediante formatos impresos u hojas de cálculo. Esto ocasiona retrasos en la atención y dificulta que la ciudadanía consulte el estado de su trámite. Los trámites que se generan a diario son demasiados para contenerlos en componentes físicos limitantes, a su vez, los trámites suelen ejecutarse paralelamente. Esto ocasiona un “cuello de botella” alargando los tiempos de espera para los usuarios.

Procesos claves a sistematizar: Para solucionar el desorden de forma sencilla, el sistema inicial debe abarcar únicamente el registro de la persona solicitante, la generación de la solicitud de licencia y la verificación de la documentación o requisitos. A partir de esta abstracción, podemos replicarlos en los demás trámites que se tienen pensando sistematizar, con ayuda de las características de la Programación Orientada a Objetos: herencia, polimorfismo, abstracción, interfaces, etc.

Nos enfocaremos solamente en elementos necesarios dentro de la ventanilla de atención ciudadana:

- **Ciudadanía:** Son las personas que solicitan el trámite. En el sistema, esto se puede representar con la clase “Ciudadano”.
- **Gestión del trámite:** Son los elementos administrativos más básicos que requerirá el sistema para registrar el proceso. En el sistema, esto se podría representar con las clases “SolicitudLicencia” y “Requisito”.

CLASE	ATRIBUTOS	MÉTODOS
Ciudadano	nombre: String curp: String telefono: String	registrarDatos() mostrarInformacion()
SolicitudLicencia	folio: int tipoLicencia: String aprobacion: Boolean	procesarSolicitud() consultarEstado()
Requisito	nombreDocumento: String entregaDoc: Boolean	entrega()

Relaciones entre clases

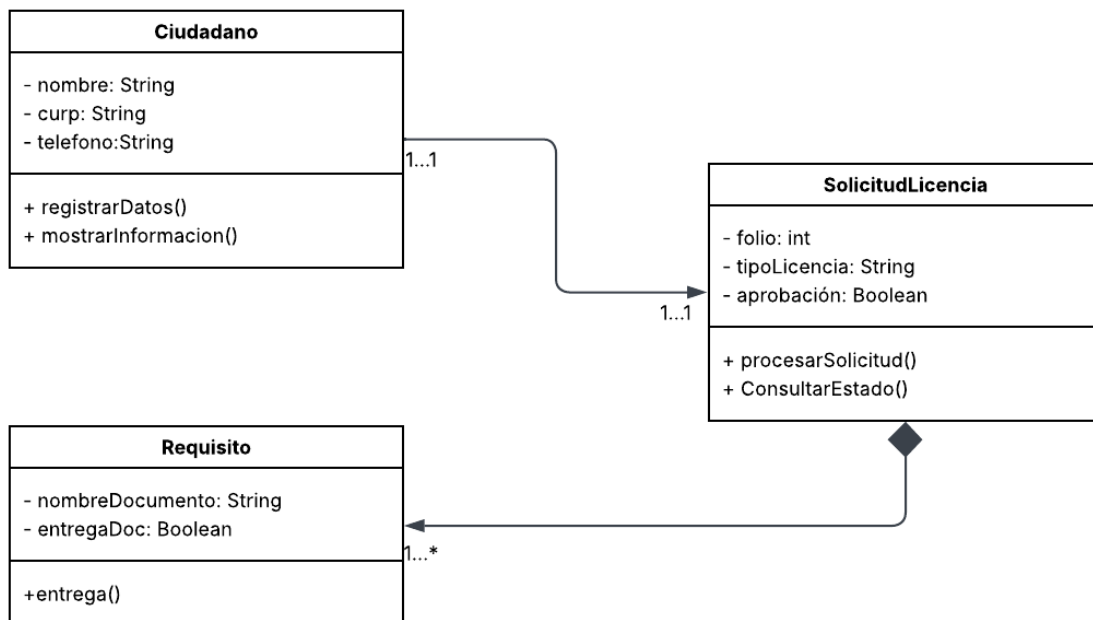
a. Relación Ciudadano - SolicitudLicencia

- Tipo de relación: Asociación directa (1..1)
- Un ciudadano es quién acude a ventanilla para registrar una solicitud de licencia, esa solicitud sólo le pertenece a ese mismo ciudadano.

b. Relación SolicitudLicencia - Requisito

- Tipo de relación: Composición (1..*)
- Una solicitud de licencia requiere la revisión de varios requisitos. Si la solicitud es rechazada o cancelada, la validación de esos requisitos pierde sentido en el flujo del sistema.

Diagrama UML



CONCLUSIÓN

Con la Programación Orientada a Objetos, podemos abstraer y modelar procesos, situaciones componentes del mundo real, para realizar sistemas que nos ayuden a hacer eficientes los trabajos de las personas, otorgándoles herramientas para sus tareas.

POO se basa en plantillas llamadas clases, éstas son abstraídas del mundo real. Las clases contienen atributos (características de la clase) y métodos (acciones que la clase puede hacer).

Por otro lado, se tiene algunos principios que son aplicables dentro de las clases:

- Encapsulamiento: protege el acceso a los atributos de la clase, para salvaguardar los detalles y, que se tenga conocimiento de lo necesario para la interacción entre las clases. Los tipos que encontramos en el encapsulamiento son:
 - public – se puede acceder al atributo, desde cualquier parte del código.
 - private – es accesible solo desde la misma clase.
 - protected – accesible desde la misma clase, las clases que se generen a partir de la principal y las clases en el mismo paquete.
- Abstracción: oculta los detalles con que se construye, definiendo y estructura solo lo esencial de una clase. Es como el concepto de “caja negra” no sabemos como lo hace, pero nos quedamos con el resultado.
- Interfaces: en la clase de tipo interfaz, solo declaramos (escribimos) los métodos, pero no escribimos su funcionalidad, principalmente se usan para realizar polimorfismo.
- Polimorfismo: un método de una clase se puede usar en una o más clases subsecuentes, pero con resultados distintos dependiendo el objetivo de la implementación y las variables que se le puedan pasar.
- Herencia: una clase hija, hereda de la clase padre los atributos y métodos para poderlos reutilizar. Con la opción de poder complementar con características específicas de la clase hija. Esto nos ayuda a reutilizar código.

Para esta práctica, se aplicaron los conocimientos descritos en la parte de arriba, permitiendo tener un conocimiento técnico que despeja las dudas de lo teórico. Empezamos a diseñar la parte de la licencia de manejo, con tres clases como son: ciudadano, requisitos y solicitud licencia. Aplicando un diagrama de

clases (una visión gráfica que nos muestra como está estructurado el sistema) y que nos permite organizar las clases y los comportamientos entre ellas.

Vimos también las como relacionar las clases, como son:

- Herencia: La clase hija hereda de la clase padre y se representa con una flecha vacía.
- Asociación: línea continua y nos dice que existe una relación de uso entre dos clases, unidireccional o bidireccional.
- Agregación: línea continua con rombo vacío, significa que una clase está conteniendo a otra y ambas, pueden existir sin la otra.
- Composición: línea continua con rombo lleno, significa que una clase está conteniendo a otra y ambas, pero no pueden existir sin la otra.
- Dependencia: línea discontinua, una clase depende de la otra temporalmente.
- Realización: línea discontinua con flecha vacía, es la definición de implementar una interfaz.

Aunque apenas estamos empezando el proyecto (porque tiene que abarcar más trámites, funcionalidades y características), ha ayudado a comprender los conceptos de la POO.

REFLEXIÓN DE LA ACTIVIDAD

1. **Explica como el modelado UML te permitió identificar la diferencia entre un enfoque procedural y uno orientado a objetos, especialmente en un sistema donde intervienen múltiples actores y procesos administrativos.**

Yo pude identificar las diferencias a partir de dos diagramas: el de flujo y el de UML. Mientras que en uno se tiene que hacer de forma secuencial, el otro se basa en el comportamiento y estructura de los componentes. Un sigue una serie de pasos para llegar a un fin común y el otro abstrae más características y funcionalidades del mundo real. Aunque el procedural puede ser fácil de aprender, el orientado a objetos nos permite una mejor organización y reutilización del código; algo que en el primero no podemos hacer, teniendo que escribir el mismo código las veces que lo necesitemos.

2. **¿Qué descubriste sobre la relevancia de identificar correctamente las entidades, atributos y responsabilidades del sistema antes de escribir código? Relacionado con un problema real que podría surgir si esta etapa se omite.**

Descubrí que uno debe tener en claro los componentes y las reglas del negocio del sistema que va a desarrollar, para obtener los resultados esperados y bajar los costos (tiempo, dinero, recursos). Esto nos permite hacer un plano (como en arquitectura) con un modelado y diseño acorde a lo solicitado. Sobre este diseño, uno puede plantearse opciones de mejora, que a la hora de escribir el código no ahorra costos y nos permite implementar de forma eficiente, los conceptos de POO.

3. **Describe un caso concreto del sistema donde aplicar herencia resultó útil. ¿Qué clase padre definiste y qué clases hijas surgieron? ¿Por qué esta estructura mejora el diseño?**

Para mí, el tipo de herencia se debe aplicar en las relaciones de: Persona (como clase padre), usuario y empleado (como clases hijas). Esto porque ambas son personas, pero en el sistema tienen diferentes acciones que realizar. Mientras que los usuarios solicitan y gestionan trámites, los empleados validan información; pero teniendo la mayor parte de las características como: nombre, apellido paterno, apellido materno, correo o contraseña.

4. **Analiza el papel de las interfaces o clases abstractas dentro de tu diseño. ¿Qué comportamiento común buscabas representar y por qué es útil en sistemas de trámites ciudadanos?**

Los trámites tienen algo en común, empiezan de la misma forma; con una solicitud, una cita, con ingresar los documentos de identificación jurídica de la persona o con inclusive el inicio de sesión. Pero conforme el usuario va avanzando con el proceso del trámite, empiezan a especializarse de una forma que, por ejemplo, para validar una identificación no se tiene que discriminar si es INE, pasaporte, cédula, etc, se utilizaría estructuras definidas en una abstracción; pero para

cuando quieres calcular el costo del trámite tienes que definir en las clases subsecuentes, características específicas que no comparten los trámites.

5. **Menciona un ejemplo claro de polimorfismo en tu diseño. Explica cómo este mecanismo simplifica la interacción del sistema con diferentes tipos de trámites o procesos.**

Cuando el usuario quiere que se le muestre la información de su trámite para ver su estatus, el resultado no es el mismo. Es diferente cuando es de licencia, porque de ahí puede desencadenar otros procesos o pendientes, pero si es un trámite de tener algún documento mostraría el estatus, pero con la opción de imprimir, por ejemplo, su acta de nacimiento.

6. **Desde tu perspectiva, ¿cómo podría un sistema como el que diseñaste contribuir a mejorar la experiencia de la ciudadanía? Considera aspectos como eficiencia, reducción de errores, acceso a información y transparencia.**

Primero para validar los datos, se podría hacer de una forma automática, validando la información con instituciones como RENAPO, INE, Registro Civil, a partir de servicios a sus sistemas. Que los diferentes trámites o proceso de estos sean hasta cierto punto con la misma ruta o mecánica de interacción. Encriptar los datos y resguardarlos generando llaves simétricas.

7. **Enumera tres aprendizajes clave de esta práctica y explica cómo cada uno te prepara para la etapa de implementación en JAVA.**

El polimorfismo, pensé que no se podía tomar algo definido y adecuarlo a los requerimientos específicos, pero se puede. Esto me ayuda a optimizar el tiempo en no rehacer código.

La responsabilidad única, muchas veces le cargamos "la mano" a una clase para hacerla que haga todo, pero esto nos contará trabajo para

cuando hagamos mantenimiento. Es por eso que tenemos que asignarle las responsabilidades en específico y no de más clases.

La abstracción del mundo real al del código, en ocasiones es difícil pensar en algo que vemos, tocamos y hasta olemos, como llevarlo a la programación. Pero cuando se tiene claro se generan moldes que podemos utilizar en otros proyectos.